

# Linux Server Hardening & Secure Configuration

## Final Project Report

Project Started: February 10, 2026

Last Updated: August 28, 2026

Status: Completed (Phase 4 of 4)

### 1. Executive Summary

The **Linux Server Hardening & Secure Configuration** project has been successfully transformed from a static documentation repository into a fully automated, idempotent, and production-ready DevSecOps framework. This framework systematically applies security baseline controls across critical OS subsystems while ensuring strict administrative safety, automated configuration backups, and built-in diagnostic auditing.

### 2. Architecture & Design Principles

The framework was built with safety as the primary directive. It orchestrates hardening across multiple modules using the following mechanisms:

- Idempotency:** Modules can be run repeatedly without duplicating configurations or corrupting files.
- Atomic Backups:** Before any configuration (`/etc/ssh`, `/etc/ufw`, etc.) is modified, a timestamped `.tar.gz` backup is taken, ensuring quick rollback.
- Validation-First Deployment:** Changes to critical services (e.g., SSH) are strictly validated using `sshd -t` before the daemon is reloaded.
- Dry-Run Mode:** Operators can simulate hardening (`--dry-run`) to preview the exact configuration changes without altering the host.

### 3. Hardening Phases & Applied Controls

#### Phase 1: Core Automation & Safety Foundation

Established the central framework (`hardened.sh`), logging library, and backup engine. Implemented system environment checks to enforce root privilege execution and strictly supported operating systems (Ubuntu, Debian, Kali Linux).

#### Phase 2: Access & Perimeter Security

- OpenSSH Server (02-ssh.sh):** Enforced key-based authentication (`PubkeyAuthentication yes`), disabled root login, and denied empty passwords. Safely disabled password authentication only after successfully verifying the existence of `authorized_keys`.
- Uncomplicated Firewall (03-firewall.sh):** Established default deny routing/incoming policies. Dynamically detected the active SSH port to rate-limit and allow it explicitly, ensuring the administrator is never locked out.

#### Phase 3: Advanced Defense

- Kernel & Network (04-sysctl.sh):** Hardened the IPv4/IPv6 stacks by disabling IP forwarding, disabling source routing, ignoring ICMP redirects, and enabling TCP SYN cookies to mitigate DoS attacks.
- Intrusion Prevention (05-fail2ban.sh):** Configured and activated a Fail2Ban `sshd` jail to automatically ban IP addresses exhibiting brute-force attack behavior.
- System Auditing (06-auditd.sh):** Deployed kernel-level auditing to monitor identity files (`/etc/passwd`, `/etc/shadow`) and privilege escalation vectors (`/etc/sudoers`) for unauthorized modifications.

#### Phase 4: Automated Security Scoring

- Lynis Integration (07-lynis.sh):** Integrated the Lynis auditing engine to dynamically score the system after hardening, parsing the official **Hardening Index** and logging comprehensive `.dat` audit reports into the `audit/reports/` directory.

### 4. Testing & Validation

All modules were validated against rigorous criteria:

- ShellCheck Compliance:** Adherence to standard Bash best practices, including safe variable quoting and `set -euo pipefail`.
- Environment Isolation:** Testing verified that the framework correctly aborted in non-Linux or unsupported OS environments without damage.
- Execution Confidence:** Full execution (Phase 1-4) on target VMs successfully applied configurations, activated firewalls, and retained administrative access.

### 5. Conclusion

The Linux Server Security Hardening Framework is now robust, complete, and production-ready. It provides an essential automated toolchain for DevSecOps engineers to quickly deploy, verify, and document a secure baseline across server fleets.